

Assignment - 2

DA6400: Reinforcement Learning

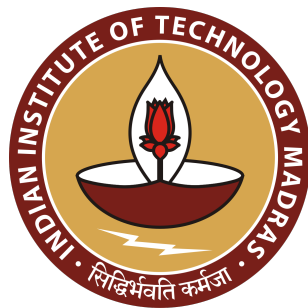
By

Ashmit Sinha and Shahid Mulla

ME21B037 and ME21B122

Course Instructor:

Prof. Balaram Ravindran



Indian Institute of Technology, Madras

March, 2025

Contents

1	Algorithms	1
1.1	Dueling-DQN and Monte-Carlo Reinforce	1
1.1.1	Dueling Deep Q-Network (Dueling DQN)	1
1.1.2	Monte Carlo REINFORCE	3
2	Implementation Specifics	5
2.1	Implementation Details - Dueling DQN	5
2.2	Implementation Details - MC REINFORCE	6
3	CartPole-v1	8
3.1	Introduction	8
3.1.1	State and Environment Details	8
3.1.2	Reward System	9
3.1.3	Transition Dynamics and Environment Settings	9
3.1.4	Training Objective	9
3.1.5	Hyperparameter Optimization	9
3.2	Results	10
3.2.1	Experimental Results - Dueling-DQN with Softmax	10
3.2.2	Comparison 1: Dueling DQN (Version 1 vs Version 2)	12
3.2.3	Experimental Results - Monte Carlo REINFORCE	13
3.2.4	Comparison 2: MC REINFORCE with & without Baseline	15
4	Acrobot-v1	16

4.1	Introduction	16
4.1.1	State and Environment Details	16
4.1.2	Reward System	17
4.1.3	Transition Dynamics and Environment Settings	17
4.1.4	Training Objective	17
4.1.5	Hyperparameter Optimization	18
4.2	Results	18
4.2.1	Experimental Results - Dueling-DQN with Softmax	18
4.2.2	Comparison 1: Dueling DQN (Version 1 vs Version 2)	20
4.2.3	Experimental Results - Monte Carlo REINFORCE	21
4.2.4	Comparison 2: MC REINFORCE with & without Baseline	23
5	Appendix - (Code Snippets & Github Link)	25
5.1	MC REINFORCE	25
5.2	Dueling DQN	27
	Credits and Resources	29

Chapter 1

Algorithms

1.1 DUELING-DQN AND MONTE-CARLO REINFORCE

In this section we present the pseudo code for **Dueling DQN** and **Monte-Carlo Reinforce** algorithms.

1.1.1 Dueling Deep Q-Network (Dueling DQN)

Algorithm 1 Dueling DQN

Initialize primary network $Q(s, a; \theta)$ and target network $Q_{\text{target}}(s, a; \theta^-)$ with weights $\theta^- \leftarrow \theta$

Initialize replay buffer $\mathcal{D}$

for each episode **do**

Initialize state s

while state s is not terminal or truncated **do**

Select action a from s using policy derived from Q (Softmax in our case)

Execute action a , observe reward r and next state s'

Store transition (s, a, r, s') in replay buffer $\mathcal{D}$

Sample random mini-batch of transitions from $\mathcal{D}$

for each sampled transition (s, a, r, s') **do**

Compute target:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-)$$

Perform gradient descent step on loss:

$$\mathcal{L}(\theta) = (y - Q(s, a; \theta))^2$$

end for

Periodically update target network: $\theta^- \leftarrow \theta$

$s \leftarrow s'$

end while

end for

Dueling DQN decomposes the action-value function into separate estimators for the state value function and the advantage function. There are two common approaches to combine these components:

- **Mean normalization (Version 1):**

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

This formulation ensures that the average advantage across all actions is zero, helping to improve learning stability.

- **Max normalization (Version 2):**

$$Q(s, a) = V(s) + \left(A(s, a) - \max_{a'} A(s, a') \right)$$

This version centers the advantage function by its maximum value, which can accelerate learning in some cases but may lead to higher variance.

The Q-values are updated by minimizing the loss between predicted Q-values and target Q-values computed using the target network:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(y - Q(s, a; \theta))^2]$$

where the target is:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-)$$

The neural network architecture used in Dueling DQN consists of an input layer corresponding to the state dimension, followed by two fully connected layers of 64 neurons each with ReLU activations for shared feature extraction. After this shared representation, the network splits into two separate streams: the value stream and the advantage stream.

The value stream processes the shared features through a fully connected layer of 256 neurons, followed by an output layer producing a single scalar value $V(s)$. Similarly, the advantage stream passes the shared features through its own fully connected layer of 256 neurons, followed by an output layer producing advantage values $A(s, a)$ for each action dimension. The final Q-values are computed by combining the outputs of the value and advantage streams.

This architectural separation allows the network to estimate the relative importance of actions more effectively, leading to improved learning stability and performance in environments with noisy or redundant action spaces.

1.1.2 Monte Carlo REINFORCE

Monte Carlo REINFORCE is a foundational policy gradient algorithm that optimizes a stochastic policy by directly estimating the gradient of the expected return using complete episodes. Unlike value-based methods, MC REINFORCE relies on sampled trajectories to update the policy parameters θ in the direction that maximizes the expected return. The gradient estimate is computed using the log-likelihood function, which avoids the need to differentiate through the environment dynamics.

Without Baseline: The policy gradient update without any baseline is:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi(a_t | s_t; \theta)$$

Here, G_t is the cumulative return from timestep t , and $\pi(a_t | s_t; \theta)$ is the probability of action a_t under the current policy parameters. This update increases the log-probability of actions that result in higher returns.

Algorithm 2 Monte Carlo REINFORCE (Without Baseline)

Initialize policy network $\pi(a|s; \theta)$ with random weights
for each episode **do**
 Generate an episode $\{(s_0, a_0, r_1), \dots, (s_T, a_T, r_{T+1})\}$ using $\pi(a|s; \theta)$
 for each timestep t in episode **do**
 Compute return: $G_t = \sum_{k=t+1}^{T+1} \gamma^{k-t-1} r_k$
 Compute gradient: $\nabla_{\theta} J(\theta) = G_t \nabla_{\theta} \log \pi(a_t | s_t; \theta)$
 Update policy: $\theta \leftarrow \theta + \alpha \gamma^t \nabla_{\theta} J(\theta)$
 end for
end for

With Baseline: To reduce variance in the gradient estimates, a baseline function $V(s_t; \Phi)$ is introduced. The updated policy gradient becomes:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - V(s_t; \Phi)) \nabla_{\theta} \log \pi(a_t | s_t; \theta)$$

Algorithm 3 Monte Carlo REINFORCE with Baseline

Initialize policy network $\pi(a|s; \theta)$ and baseline network $V(s; \Phi)$ with random weights
for each episode **do**
 Generate an episode $\{(s_0, a_0, r_1), \dots, (s_T, a_T, r_{T+1})\}$ using $\pi(a|s; \theta)$
 for each timestep t in episode **do**
 Compute return: $G_t = \sum_{k=t+1}^{T+1} \gamma^{k-t-1} r_k$
 Compute baseline: $\hat{V}(s_t; \Phi)$
 Compute gradient: $\nabla_{\theta} J(\theta) = (G_t - \hat{V}(s_t; \Phi)) \nabla_{\theta} \log \pi(a_t | s_t; \theta)$
 Update policy: $\theta \leftarrow \theta + \alpha \gamma^t \nabla_{\theta} J(\theta)$
 Compute TD(0) target: $y = r_{t+1} + \gamma \hat{V}(s_{t+1}; \Phi)$
 Update baseline: $\Phi \leftarrow \Phi - \beta \nabla_{\Phi} (\hat{V}(s_t; \Phi) - y)^2$
 end for
end for

Baseline Update using TD(0): The baseline network $V(s; \Phi)$ is trained to minimize the mean-squared error between the predicted state value and the one-step return. This is achieved using the Temporal Difference (TD) learning method with zero-step bootstrapping-TD(0).

The TD target for a given timestep t is computed as:

$$y_t = r_{t+1} + \gamma V(s_{t+1}; \Phi)$$

The loss minimized by the baseline is:

$$\mathcal{L}(\Phi) = (y_t - V(s_t; \Phi))^2$$

The parameters Φ are updated by taking a gradient descent step:

$$\Phi \leftarrow \Phi - \beta \nabla_{\Phi} \mathcal{L}(\Phi)$$

where β is the learning rate for the baseline network. This allows the baseline to learn an unbiased estimate of expected returns from states, which is used to reduce the variance in policy gradient updates.

Neural Network Architecture:

For both versions of the REINFORCE algorithm, neural networks are employed to represent the policy, while in the baseline variant, an additional value network is introduced to estimate the state-value function.

- **Policy Network:** A feedforward neural network with a single hidden layer of size 128 followed by a ReLU activation. The final layer outputs a probability distribution over actions using a Softmax function. This architecture is common to both the baseline and non-baseline versions.
- **Without Baseline:** Only the policy network is used to sample actions and update the parameters using Monte Carlo returns. The log-probabilities of selected actions are scaled by normalized returns to form the loss used for gradient ascent.
- **With Baseline:** An additional value network is introduced to reduce the variance of the policy gradient estimates. The value network shares a similar architecture with the policy network, but the final layer outputs a scalar value corresponding to the estimated state-value. The baseline (value network) is trained using TD(0), and the advantage $G_t - V(s_t)$ is used in the policy gradient update.

Chapter 2

Implementation Specifics

2.1 IMPLEMENTATION DETAILS - DUELING DQN

The Dueling DQN implementation for the CartPole environment consists of two main components: the agent class and the neural network architecture used to approximate the action-value function.

Agent Class: `Dueling_DQN_V1_softmax`

The agent interacts with the environment and uses a neural network to predict action-values. Action selection is performed using a softmax policy over the predicted Q-values to encourage exploration. Specifically, the Q-values are exponentiated and normalized to obtain action probabilities:

$$\text{Action probabilities} = \frac{\exp(Q(s, a) - \max_a Q(s, a))}{\sum_{a'} \exp(Q(s, a') - \max_{a'} Q(s, a'))}$$

Actions are sampled from this probability distribution at each step.

Neural Network Architecture: `QNetwork_model`

The neural network is designed following the Dueling DQN architecture, which separates the estimation of the state value function $V(s)$ and the advantage function $A(s, a)$.

The architecture consists of:

- **Input layer:** Corresponding to the state dimension of the environment.
- **Shared fully connected layers:**
 - Two layers of 64 neurons each with ReLU activation.
- **Separate streams:**
 - **Value stream:**
 - * Fully connected layer with 256 neurons and ReLU activation.
 - * Output layer producing a scalar value $V(s)$.
 - **Advantage stream:**
 - * Fully connected layer with 256 neurons and ReLU activation.
 - * Output layer producing advantage values $A(s, a)$ for each possible action.

The final Q-values are computed by combining the outputs of the value and advantage streams. In this implementation, two versions of aggregation strategies have been explored:

Version 1: Mean Aggregation (Standard) This is the conventional approach for Dueling DQN, where the mean of the advantage values across all actions is subtracted to normalize the advantages:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

This formulation ensures that the average advantage across all actions is zero, which helps to stabilize learning and prevents the advantage function from dominating the value function.

Version 2: Max Aggregation (Alternative) In this variant, the maximum of the advantage values is subtracted instead of the mean:

$$Q(s, a) = V(s) + \left(A(s, a) - \max_{a'} A(s, a') \right)$$

Using the maximum advantage for normalization can lead to faster exploitation of the best actions in certain environments. However, this approach may introduce higher variance and instability in training, as it amplifies the difference between the best and suboptimal actions.

Both versions were implemented and compared to observe their impact on learning stability and overall performance in the CartPole environment.

2.2 IMPLEMENTATION DETAILS - MC REINFORCE

The MC REINFORCE implementation for both the environments includes two variants: one without a baseline and one with a state-value baseline. Each variant consists of an agent class and its corresponding neural network architecture.

Agent Class: REINFORCEAgent (Without Baseline)

This agent uses a policy network to sample actions based on a categorical distribution defined by the network’s output probabilities. At the end of each episode, Monte Carlo returns G_t are computed and used to update the policy.

The agent stores log-probabilities of the actions taken and rewards received during the episode. After the episode ends, it calculates the discounted returns and normalizes them. The loss function is constructed as:

$$\mathcal{L}_{\text{policy}} = - \sum_t \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

where G_t is the return from time step t . The parameters are updated via gradient ascent to maximize expected return.

Agent Class: REINFORCEWithBaselineAgent (With Baseline)

This variant introduces a value network as a baseline to reduce the variance of the policy gradient estimates. The agent additionally stores the visited states during the episode. After computing Monte Carlo returns, it estimates the advantage as:

$$A_t = G_t - V(s_t; \Phi)$$

The policy is updated using this advantage:

$$\mathcal{L}_{\text{policy}} = - \sum_t \log \pi_{\theta}(a_t | s_t) \cdot A_t$$

Meanwhile, the value network is trained using a mean squared error loss:

$$\mathcal{L}_{\text{value}} = \sum_t (G_t - V(s_t; \Phi))^2$$

The baseline is updated using the TD(0) method, which, in this episodic setting, corresponds to regression toward the return G_t as a supervised target.

Neural Network Architectures

- **Policy Network (Both Variants):** A simple two-layer feedforward network with:
 - **Input layer:** Matches the observation dimension of the environment.
 - **Hidden layer:** 128 neurons with ReLU activation.
 - **Output layer:** Outputs a probability distribution over actions using Softmax.
- **Value Network (Baseline Variant Only):**
 - **Input layer:** Matches the observation dimension.
 - **Hidden layer:** 128 neurons with ReLU activation.
 - **Output layer:** Outputs a single scalar representing the estimated state value $V(s)$.

The architecture is intentionally lightweight to suit the simplicity and small state space of both the environments. The separation of the policy and value networks allows independent optimization of action selection and state-value estimation, facilitating better stability in training when using a baseline.

Chapter 3

CartPole-v1

3.1 INTRODUCTION

The first setting for this assignment is the **CartPole** environment, as shown in Figure 1. The agent controls a cart that moves along a **one-dimensional track**, balancing a pole attached to it by a hinge as shown in Figure 2.1.



Figure 3.1: CartPole Environment

3.1.1 State and Environment Details

- **State Space:** The state is represented by a **4-dimensional** continuous vector:
 - Cart Position x : $-4.8 \leq x \leq 4.8$
 - Cart Velocity v : $-\infty \leq v \leq \infty$
 - Pole Angle θ : $-0.418 \text{ rad } (-24^\circ) \leq \theta \leq 0.418 \text{ rad } (24^\circ)$
 - Pole Angular Velocity ω : $-\infty \leq \omega \leq \infty$
- **Action Space:** The agent has **two available actions** at each time step:
 - **Push Left (0):** Applies force to the left.
 - **Push Right (1):** Applies force to the right.
- **Goal:** The goal of the agent is to balance the pole upright for as long as possible.
- **Start State:** All observations are assigned a uniformly random value in **(-0.05, 0.05)**

3.1.2 Reward System

- The agent receives a **reward of +1** for every time step it successfully balances the pole.
- The episode **ends** if either of the following conditions is met:
 - The angle of the pole $|\theta| > 12^\circ$ from vertical.
 - The cart moves too far from the center $|x| > 2.4$.
 - **Truncation:** Episode length is greater than 500

3.1.3 Transition Dynamics and Environment Settings

- **Physics-based transitions:** The next state is determined by simulating the cart's movement using **Newtonian mechanics**.
- **Gravity:** The environment incorporates gravitational effects that cause the pole to fall unless countered by appropriate forces.
- **No stochasticity:** The transitions are **deterministic** given an action.

3.1.4 Training Objective

Our goal in this assignment is to train an **RL agent** to control the cart optimally and **maximize cumulative reward** while balancing the pole. We experiment with different algorithms, hyperparameters, and variations of the environment to determine the best-performing agent.

3.1.5 Hyperparameter Optimization

We ran a WandB search over the hyperparameter space (~ 5 runs) and used that to get an estimate of ideal hyperparams and use it for actual training purposes. The following was the space of the hyperparameters that were tuned:

Dueling DQN using Softmax

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial temperature (τ_o): $\{1.0, 5.0, 10.0\}$
- Final temperature (τ_f): $\{0.1, 0.5, 1.0\}$
- Decay rate (τ_d): $\{0.99, 0.995, 0.999\}$

3.2 RESULTS

The following section presents the results of two algorithms Dueling-DQN with Softmax and Monte Carlo REINFORCE applied to solving the CartPole-v1 environment.

3.2.1 Experimental Results - Dueling-DQN with Softmax

We landed on a set of hyperparameters after performing a sweep on Weights & Biases (WandB). We conducted a Bayesian search over the hyperparameter space for a total of 5000 episodes for each combination and used regret minimization as a metric to pick the hyperparameter configurations. Coincidentally, the best set of hyper-parameters we arrived at worked very well for both version 1 and version 2.

Best Hyperparameter Configurations

In our experiments with the CartPole environment, we plot avg. rolling reward, we run the best set of hyper-parameters for a total of **5,000 episodes** averaged over **5 seed values** to account for stochasticity.

Rank	Initial τ	Tau Decay	Final τ	Learning Rate
1	5.0	0.995	0.1	1e-3

Table 3.1: Best hyper-parameter configurations for Dueling DQN Version 1 and Version 2.

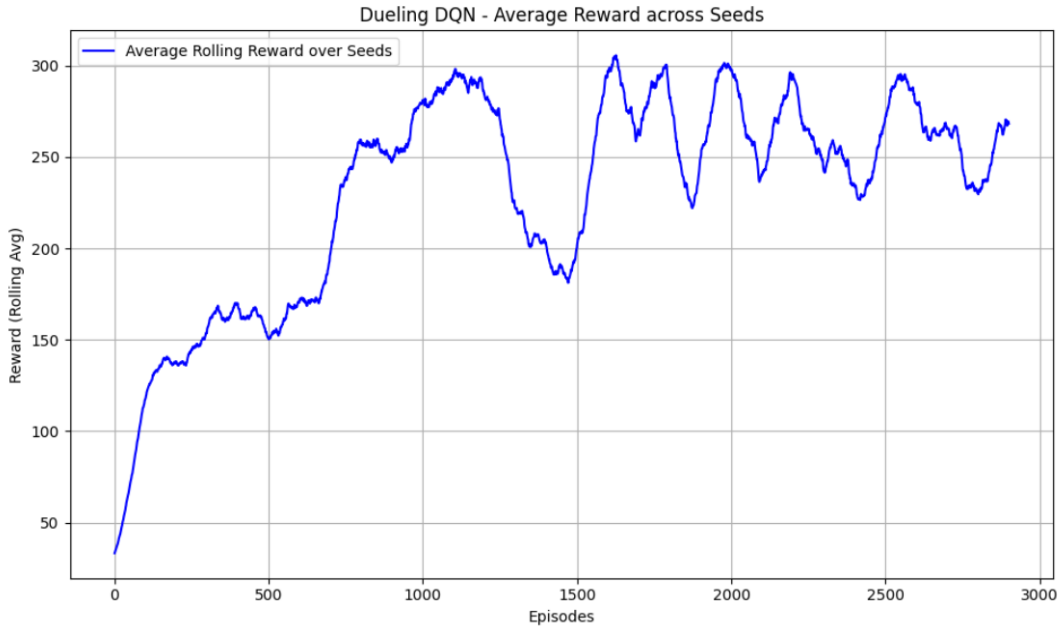


Figure 3.2: Rewards vs Episodes for Dueling DQN - Version 1

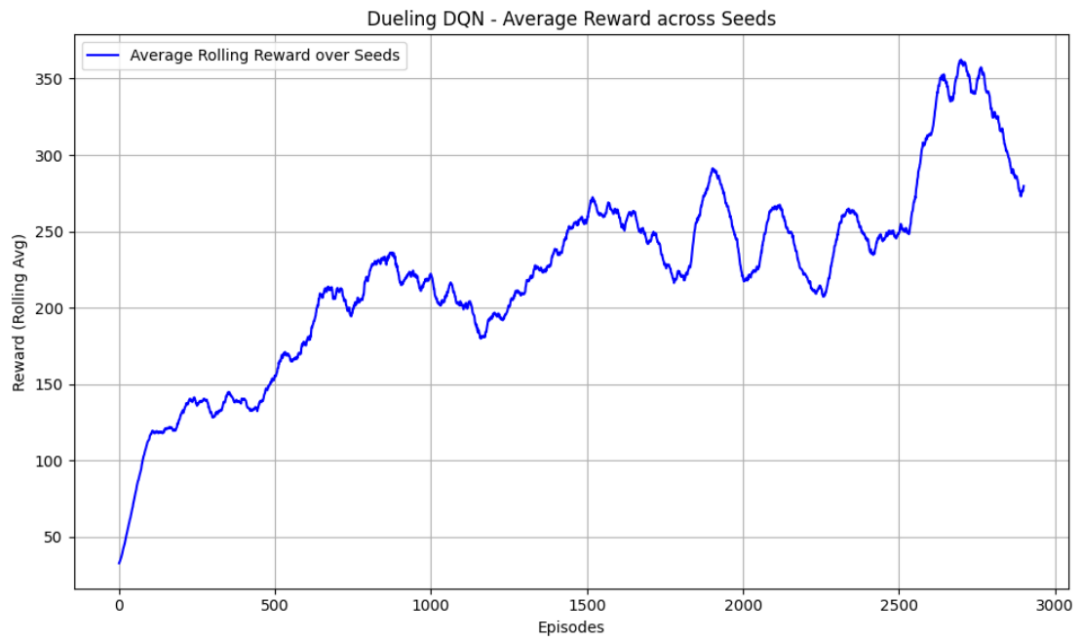


Figure 3.3: Rewards vs Episodes for Dueling DQN - Version 2

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters:

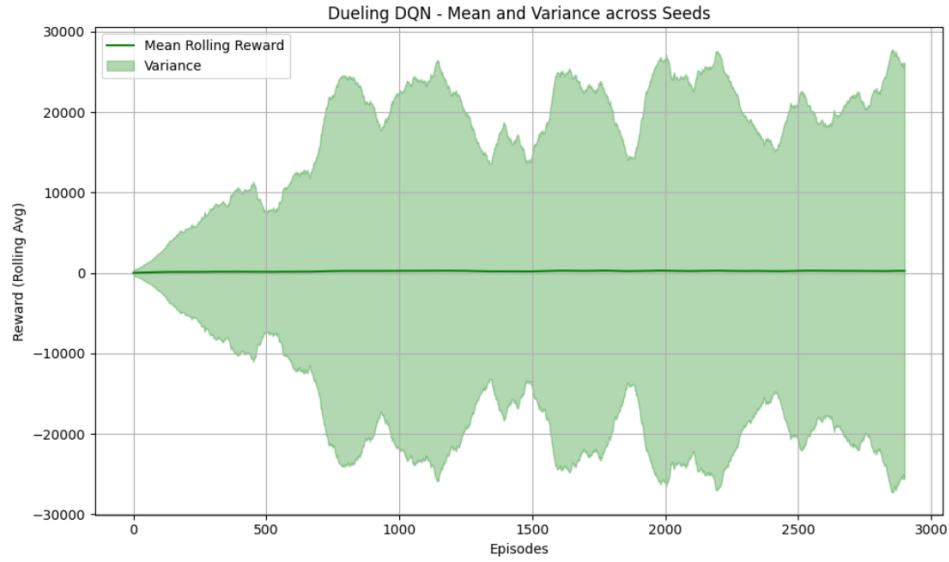


Figure 3.4: mean and Variance plots for Dueling DQN - Version 1

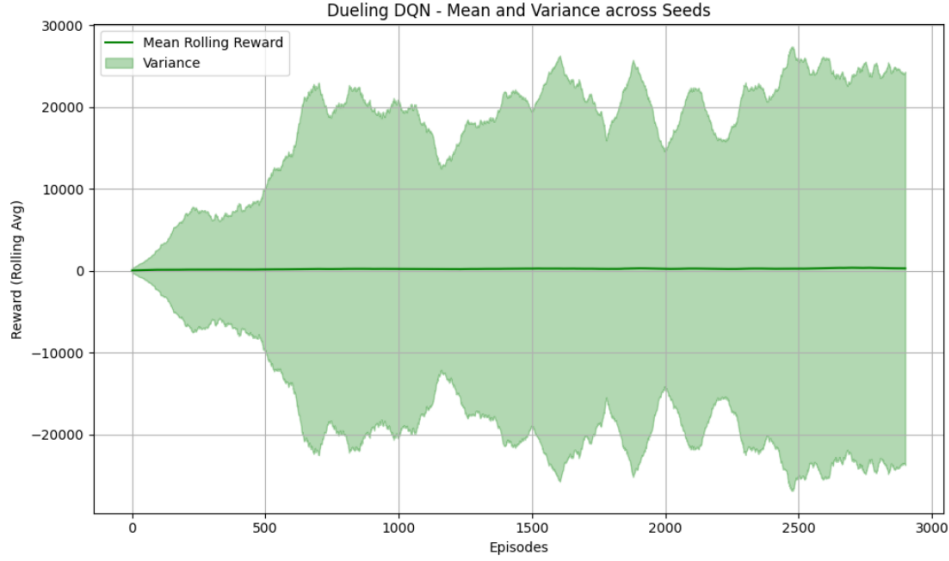


Figure 3.5: mean and Variance plots for Dueling DQN - Version 2

3.2.2 Comparison 1: Dueling DQN (Version 1 vs Version 2)

Finally, we compare the performance of Dueling DQN Version 1 and version 2.

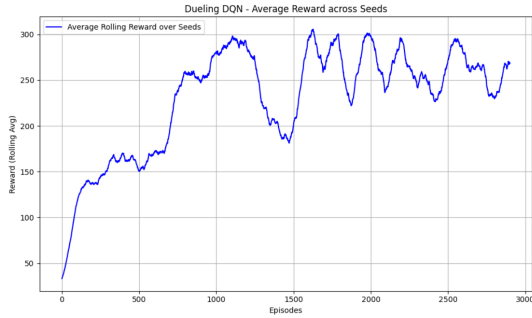


Figure 3.6: Version 1

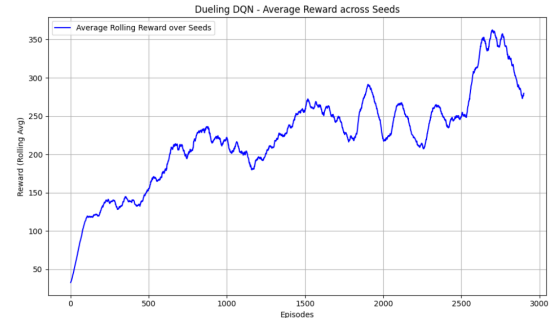


Figure 3.7: Version 2

Inference: Dueling DQN Version 1 vs. Version 2

Version 1 uses mean normalization of the advantage function, while Version 2 uses max normalization.

The following insights were drawn from the experimental results:

- **Training Stability:** Version 1 exhibited smoother and more stable training progress. In contrast, Version 2 showed faster initial gains but experienced noticeable fluctuations and instability over time.
- **Speed of Convergence:** Version 2 converged faster in the early episodes, likely due to its emphasis on high-advantage actions. However, this came at the cost of increased variability.
- **Variance:** Both versions exhibit similar trends for variance graph showing similar trends with Variance of **Version 2** being slightly higher compared to **Version 1** at the end.

3.2.3 Experimental Results - Monte Carlo REINFORCE

For the CartPole environment, we implemented the Monte Carlo REINFORCE algorithm with a fixed set of hyperparameters, without performing any hyperparameter optimization or search. The hyperparameters we initially selected worked well, and there were not many parameters to search for as the only ones were learning rates.

Hyperparameter Settings Used

The following hyperparameter values were used consistently across all runs:

Policy LR (α)	Value LR (β)	Discount Factor (γ)	Hidden Layer Size
1e-3	1e-3	0.99	128

Table 3.2: Fixed hyperparameter configuration for MC REINFORCE on CartPole.

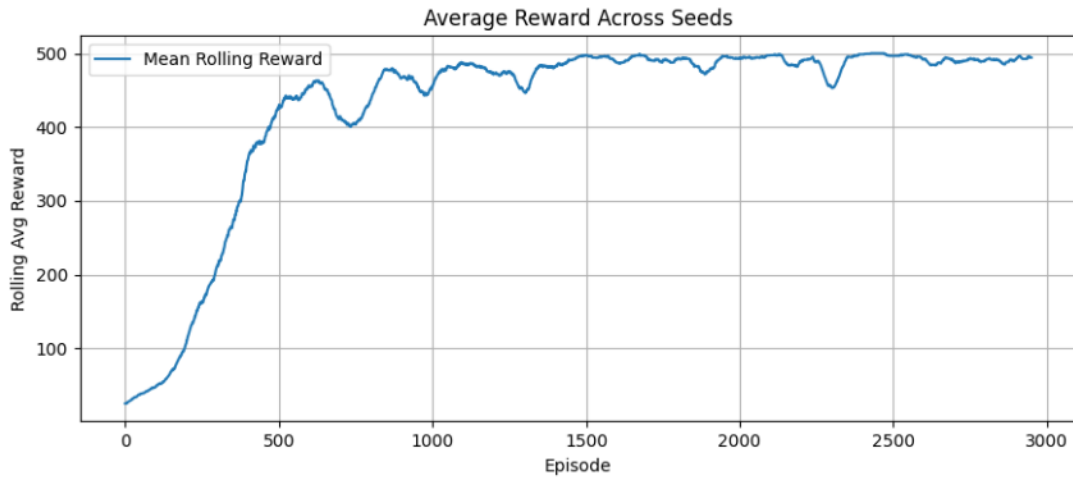


Figure 3.8: Rewards Plot for MC REINFORCE without Baseline on CartPole

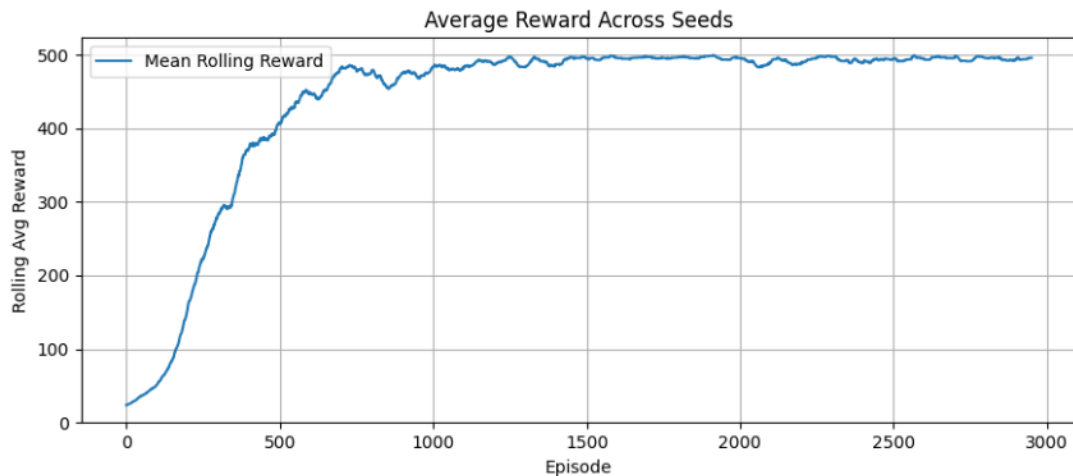


Figure 3.9: Rewards Plot for MC REINFORCE with Baseline on CartPole

Mean and Variance Plots:

We also present the mean and variance plots to analyze the consistency and variance in returns across training runs. These are averaged over **5 random seeds**.

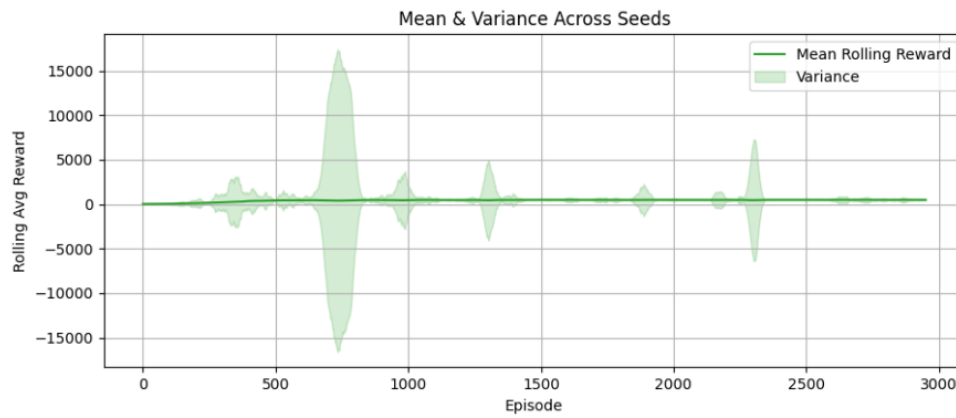


Figure 3.10: Mean and Variance of Returns for MC REINFORCE without Baseline on CartPole

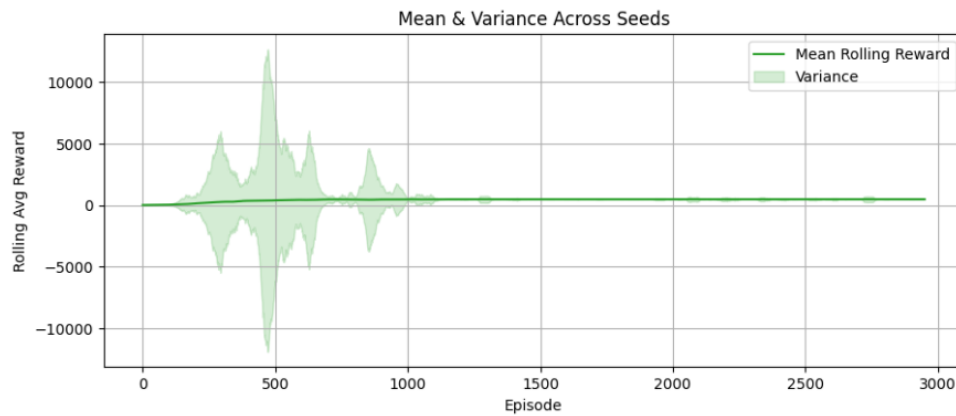


Figure 3.11: Mean and Variance of Returns for MC REINFORCE with Baseline on CartPole

3.2.4 Comparison 2: MC REINFORCE with & without Baseline

We now compare the performance of MC REINFORCE with and without the Baseline.

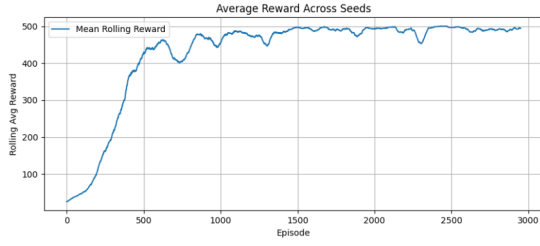


Figure 3.12: Without Baseline

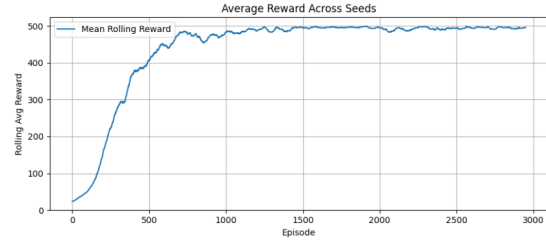


Figure 3.13: With Baseline

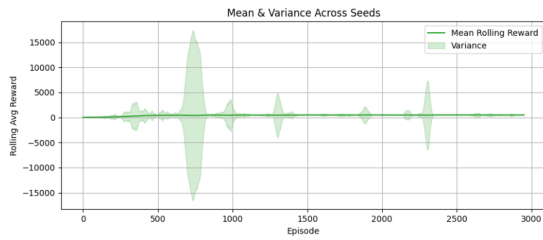


Figure 3.14: Variance without Baseline

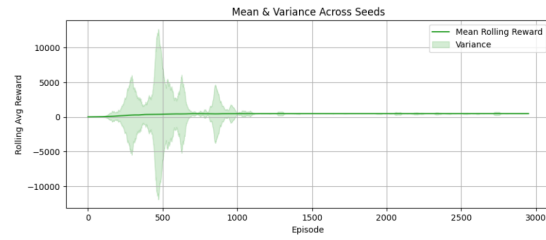


Figure 3.15: Variance with Baseline

Inference: MC REINFORCE with & without Baseline

The following are the main differences observed between the two algorithms based on the plots above:

- **Stability:** MC REINFORCE demonstrates relatively lower stability without the baseline based on the plots, even after converging we can see the rewards dropping to 450 randomly. Whereas, with the Baseline, we can see the rewards plot being more stable.
- **Convergence Pattern:** While both versions of the algorithm learn a reasonably good policy, they also converge after a similar number of episodes..
- **Variance:** The variance for both versions decreases over time, but the version with the Baseline converges to 0 and remains stable there, leading to less disturbances/instability in rewards. Without the Baseline, the variance takes approximately same number of episodes to converge to 0, but remains unstable even after converging, leading to instability in reward plots.

Chapter 4

Acrobot-v1

4.1 INTRODUCTION

The second setting for this assignment is the **Acrobot** environment, as shown in Figure 4.1. The agent controls a **two-link pendulum system**, where only the second joint (elbow) is actuated. The objective is to swing the end of the lower link to a target height by applying torques at the elbow joint, as illustrated in Figure 2.1.



Figure 4.1: Acrobot Environment

4.1.1 State and Environment Details

- **State Space:** The state is a **6-dimensional** continuous vector describing the system configuration:
 - $\cos(\theta_1)$: Cosine of the angle of the first link.
 - $\sin(\theta_1)$: Sine of the angle of the first link.
 - $\cos(\theta_2)$: Cosine of the angle of the second link.
 - $\sin(\theta_2)$: Sine of the angle of the second link.
 - $\dot{\theta}_1$: Angular velocity of the first link.
 - $\dot{\theta}_2$: Angular velocity of the second link.

- **Action Space:** The agent has **three discrete actions**:
 - **Action 0:** Apply a torque of -1 to the second joint.
 - **Action 1:** Apply a torque of 0 (no force).
 - **Action 2:** Apply a torque of +1 to the second joint.
- **Goal:** The goal is to swing the end of the pendulum (the tip of the second link) above a specified height (typically the height of the first joint).
- **Start State:** The initial joint angles and velocities are randomly initialized within a small range close to zero.

4.1.2 Reward System

- The agent receives a **reward of -1** at every time step until the goal is reached.
- This makes the task a **minimum-time problem**, where the agent is incentivized to solve the task in as few steps as possible.
- The episode **ends** when:
 - The end-effector (tip of the second link) reaches a height above a fixed threshold (i.e., $y > 1.0$ in world coordinates).
 - **Truncation:** The episode reaches the maximum step limit (typically 500 steps).

4.1.3 Transition Dynamics and Environment Settings

- **Physics-based transitions:** The motion of the acrobot is simulated using **rigid body dynamics**.
- **Underactuation:** Only the second joint is actuated, which makes the control problem more complex due to the underactuated nature of the system.
- **Deterministic transitions:** Given the current state and action, the next state is deterministically computed using physics simulation.

4.1.4 Training Objective

Our goal in this setting is to train an **RL agent** to efficiently swing the acrobot's links and reach the target height in the fewest time steps. The challenge lies in learning the proper sequence of torques that exploit the system's nonlinear dynamics.

4.1.5 Hyperparameter Optimization

We ran a WandB search over the hyperparameter space (~ 5 runs) and used that to get an estimate of ideal hyperparams and use it for actual training purposes. The following was the space of the hyperparameters that were tuned:

Dueling DQN using Softmax

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial temperature (τ_o): $\{1.0, 5.0, 10.0\}$
- Final temperature (τ_f): $\{0.1, 0.5, 1.0\}$
- Decay rate (τ_d): $\{0.99, 0.995, 0.999\}$

4.2 RESULTS

The following section presents the results of two algorithms Dueling-DQN with Softmax and Monte Carlo REINFORCE applied to solving the Acrobot-v1 environment.

4.2.1 Experimental Results - Dueling-DQN with Softmax

The following section presents the results of two algorithms Dueling-DQN with Softmax and Monte Carlo REINFORCE applied to solving the Acrobot-v1 environment.

Best Hyperparameter Configurations

In our experiments with the CartPole environment, we plot avg. rolling reward, we run the best set of hyper-parameters for a total of **5,000 episodes** averaged over **5 seed values** to account for stochasticity.

Rank	Initial τ	Tau Decay	Final τ	Learning Rate
1	5.0	0.995	0.1	1e-3

Table 4.1: Best hyper-parameter configurations for Dueling DQN Version 1 and Version 2.

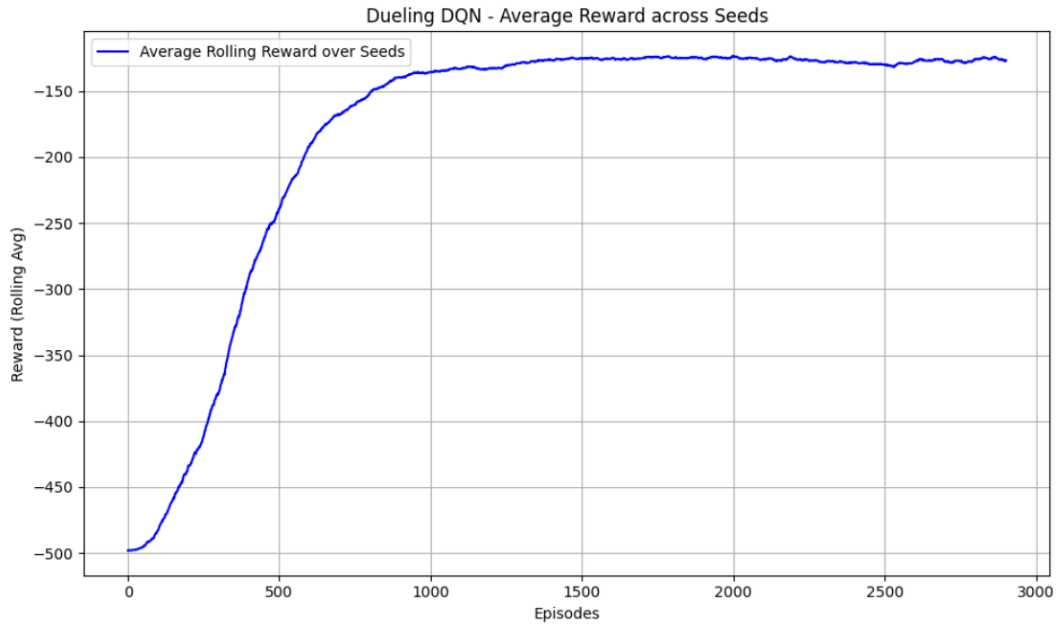


Figure 4.2: Rewards vs Episodes for Dueling DQN - Version 1

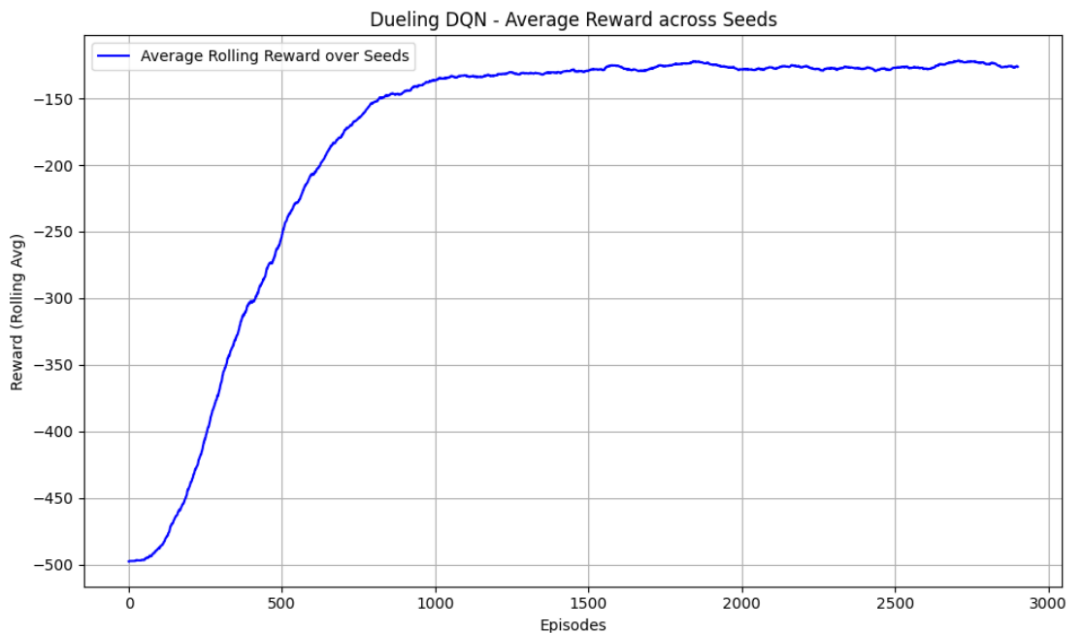


Figure 4.3: Rewards vs Episodes for Dueling DQN - Version 2

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters:

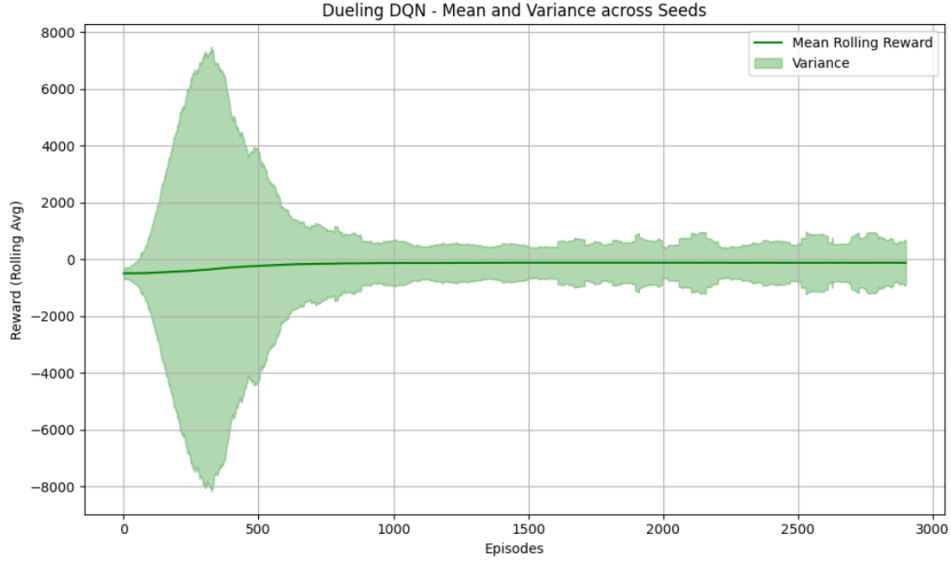


Figure 4.4: mean and Variance plots for Dueling DQN - Version 1

4.2.2 Comparison 1: Dueling DQN (Version 1 vs Version 2)

Finally, we compare the performance of Dueling DQN Version 1 and version 2.

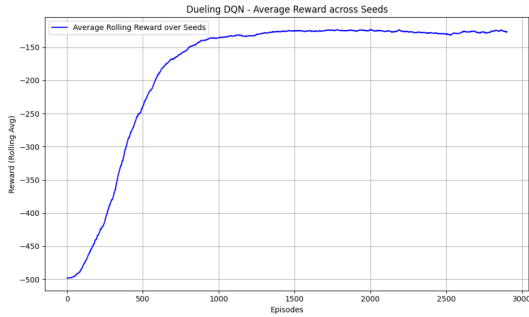


Figure 4.6: Version 1

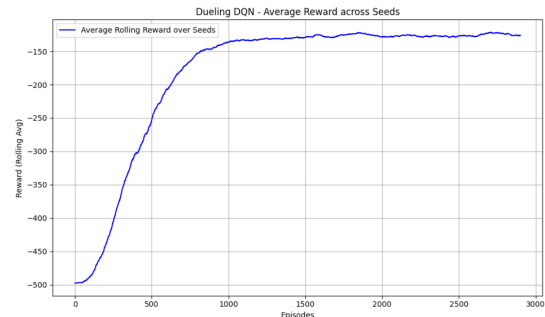


Figure 4.7: Version 2

Inference: Dueling DQN Version 1 vs. Version 2

Version 1 uses mean normalization of the advantage function, while Version 2 uses max normalization.

The following insights were drawn from the experimental results:

- **Training Stability:** Version 1 is slightly more stable compared to version 2 although the difference is very small.
- **Speed of Convergence:** Both the algorithms roughly show similar convergence speeds. They converge to around **-120**
- **Variance:** Both the versions initially see a rise in their variance followed by a sharp drop when it finally converges. Again both algorithms show similar trends for variance with no significant differences.

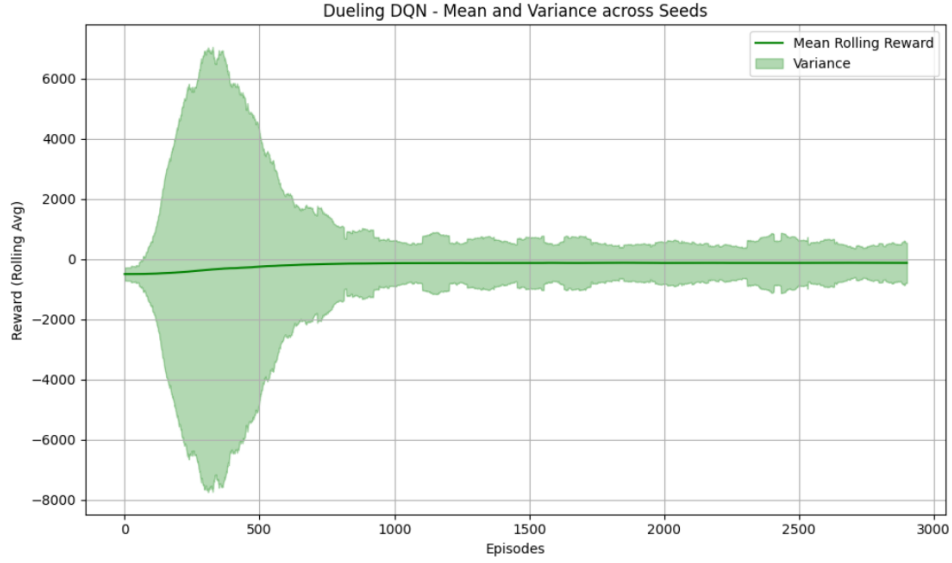


Figure 4.5: mean and Variance plots for Dueling DQN - Version 2

4.2.3 Experimental Results - Monte Carlo REINFORCE

For the Acrobot environment, we implemented the Monte Carlo REINFORCE algorithm with a fixed set of hyperparameters, without performing any hyperparameter optimization or search as the hyperparameters we tried for the first time worked perfectly, and there were not many parameters to search for as the only parameters were learning rates.

Hyperparameter Settings Used

The following hyperparameter values were used consistently across all runs:

Policy LR (α)	Value LR (β)	Discount Factor (γ)	Hidden Layer Size
1e-3	1e-3	0.99	128

Table 4.2: Fixed hyperparameter configuration for MC REINFORCE on Acrobot.

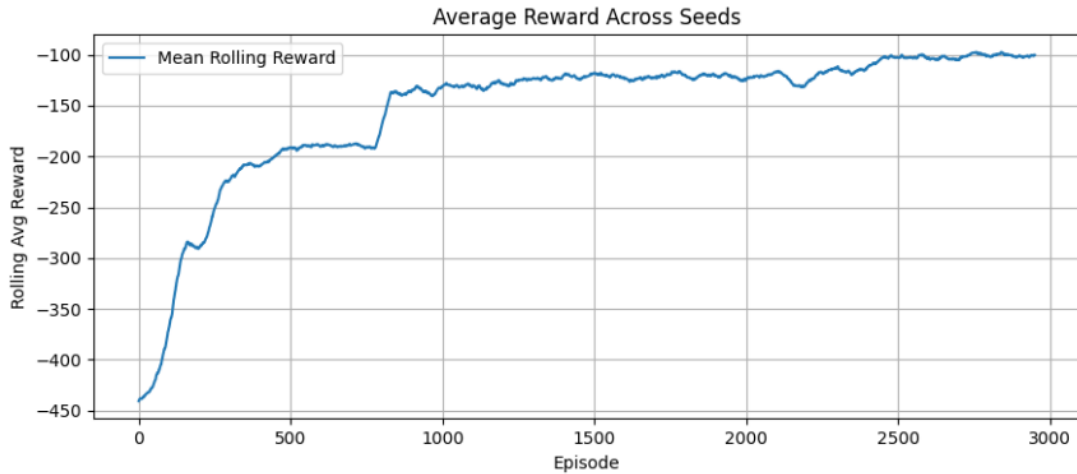


Figure 4.8: Rewards Plot for MC REINFORCE without Baseline on Acrobot

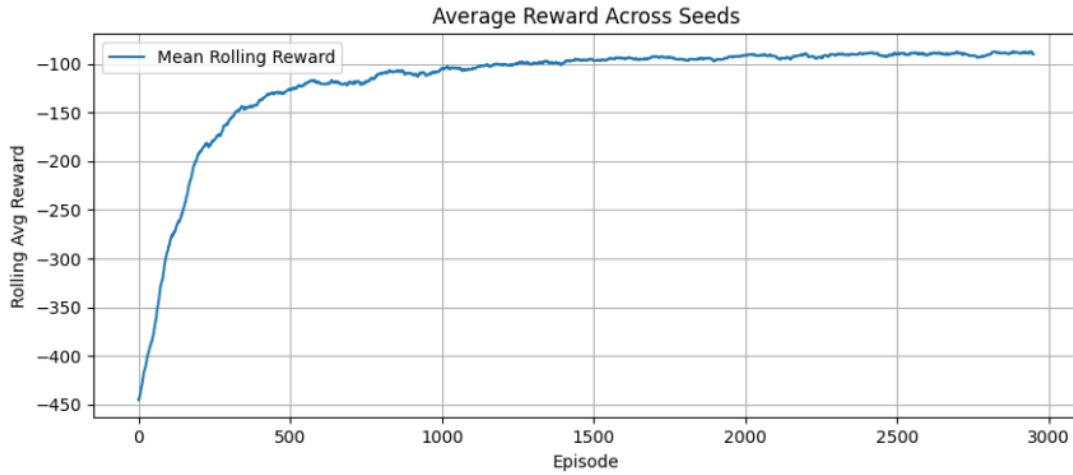


Figure 4.9: Rewards Plot for MC REINFORCE with Baseline on Acrobot

Mean and Variance Plots:

We also present the mean and variance plots to analyze the consistency and variance in returns across training runs. These are averaged over **5 random seeds**.

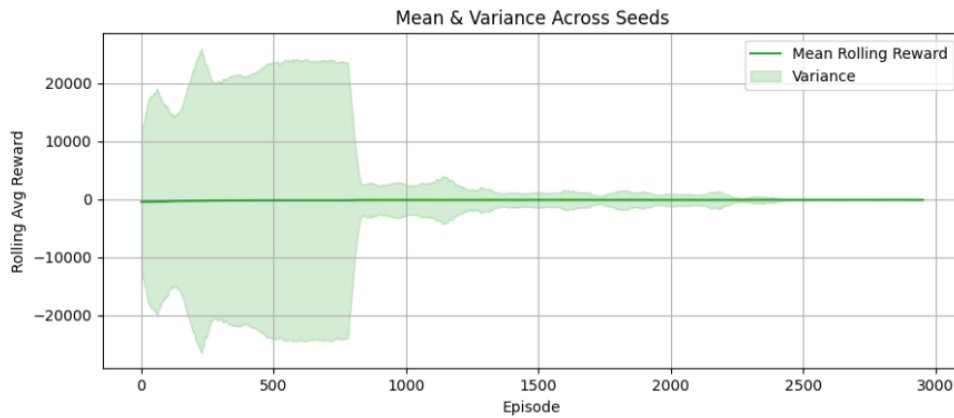


Figure 4.10: Mean and Variance of Returns for MC REINFORCE without Baseline on Acrobot

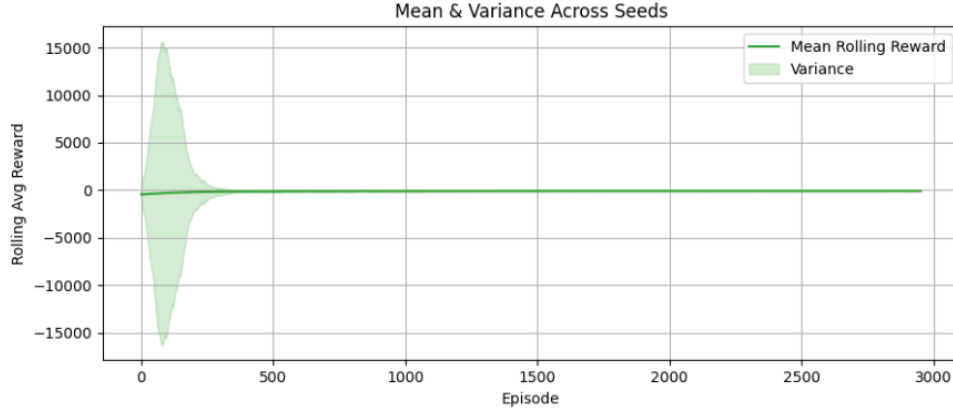


Figure 4.11: Mean and Variance of Returns for MC REINFORCE with Baseline on Acrobot

4.2.4 Comparison 2: MC REINFORCE with & without Baseline

We now compare the performance of MC REINFORCE with & without Baseline.

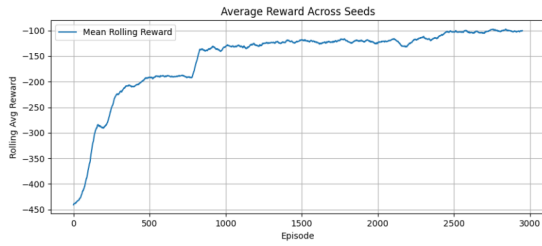


Figure 4.12: Version 1

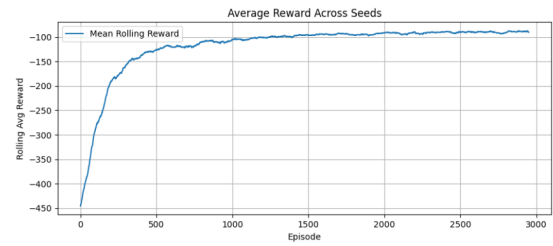


Figure 4.13: Version 2

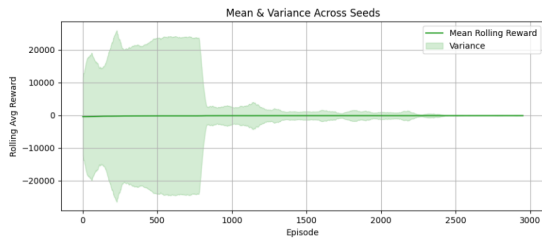


Figure 4.14: Version 1

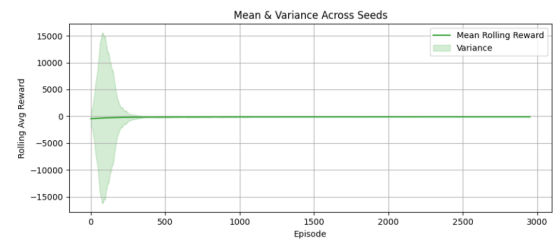


Figure 4.15: Version 2

Inference: MC REINFORCE Training Behavior

The following are the main differences observed between the two algorithms based on the plots above:

- **Stability:** MC REINFORCE demonstrates relatively lower stability without the baseline based on the plots as it converges at around 900 episodes, whereas we can see that MC REINFORCE with the Baseline converged much earlier at around 500 ~ 600 episodes & remains stable there.
- **Convergence Pattern:** While both the algorithm learns a reasonably good policy, the one with the Baseline converges earlier.

- **Variance:** Variance for the the algorithm converges to 0, but MC REINFORCE with the baseline makes the variance converge to 0 much earlier, even before 500 episodes. Whereas, the one without the Baseline took more than 1500 episodes to make the variance converge to 0.

Chapter 5

Appendix - (Code Snippets & Github Link)

5.1 MC REINFORCE

```
class PolicyNetwork(nn.Module):
    def __init__(self, obs_dim, action_dim, hidden_size=128):
        super(PolicyNetwork, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, action_dim),
            nn.Softmax(dim=-1)
        )

    def forward(self, x):
        return self.net(x)
```

Figure 5.1: MC REINFORCE without Baseline: Policy Neural Network

```
class REINFORCEAgent:
    def __init__(self, obs_dim, action_dim, hidden_size=128, lr=1e-3, gamma=0.99):
        self.gamma = gamma
        self.policy_net = PolicyNetwork(obs_dim, action_dim, hidden_size)
        self.optimizer = optim.Adam(self.policy_net.parameters(), lr=lr)

        self.log_probs = []
        self.rewards = []

    def select_action(self, state):
        state = torch.FloatTensor(state).unsqueeze(0)
        probs = self.policy_net(state)
        dist = torch.distributions.Categorical(probs)
        action = dist.sample()
        log_prob = dist.log_prob(action)
        self.log_probs.append(log_prob)
        return action.item()

    def store_reward(self, reward):
        self.rewards.append(reward)

    def update_policy(self):
        returns = []
        G = 0
        for r in reversed(self.rewards):
            G = r + self.gamma * G
            returns.insert(0, G)

        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + 1e-8)

        policy_loss = []
        for log_prob, G in zip(self.log_probs, returns):
            policy_loss.append(-log_prob * G)

        self.optimizer.zero_grad()
        loss = torch.stack(policy_loss).sum()
        loss.backward()
        self.optimizer.step()

        self.log_probs.clear()
        self.rewards.clear()

    def update_policy(self):
        returns = []
        G = 0
        for r in reversed(self.rewards):
            G = r + self.gamma * G
            returns.insert(0, G)

        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + 1e-8)

        policy_loss = []
        for log_prob, G in zip(self.log_probs, returns):
            policy_loss.append(-log_prob * G)

        self.optimizer.zero_grad()
        loss = torch.stack(policy_loss).sum()
        loss.backward()
        self.optimizer.step()

        self.log_probs.clear()
        self.rewards.clear()

        return loss.item()
```

Figure 5.2: MC REINFORCE without Baseline: Algorithm Class

```

class PolicyNetwork(nn.Module):
    def __init__(self, obs_dim, action_dim, hidden_size=128):
        super(PolicyNetwork, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(obs_dim, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, action_dim),
            nn.Softmax(dim=-1)
        )

    def forward(self, x):
        return self.model(x)

```

Figure 5.3: MC REINFORCE with Baseline: Policy Neural Network

```

class ValueNetwork(nn.Module):
    def __init__(self, obs_dim, hidden_size=128):
        super(ValueNetwork, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(obs_dim, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 1)
        )

    def forward(self, x):
        return self.model(x).squeeze(-1)

```

Figure 5.4: MC REINFORCE with Baseline: Value Function Neural Network

```

class REINFORCEwithBaselineAgent:
    def __init__(self, obs_dim, action_dim, hidden_size=128, lr_pi=1e-3, lr_v=1e-3, gamma=0.99):
        self.gamma = gamma

        self.policy_net = PolicyNetwork(obs_dim, action_dim, hidden_size)
        self.value_net = ValueNetwork(obs_dim, hidden_size)

        self.policy_optimizer = optim.Adam(self.policy_net.parameters(), lr=lr_pi)
        self.value_optimizer = optim.Adam(self.value_net.parameters(), lr=lr_v)

        self.log_probs = []
        self.rewards = []
        self.states = []

    def select_action(self, state):
        state = torch.FloatTensor(state)
        probs = self.policy_net(state)
        dist = torch.distributions.Categorical(probs)
        action = dist.sample()
        self.log_probs.append(dist.log_prob(action))
        self.states.append(state)
        return action.item()

    def store_reward(self, reward):
        self.rewards.append(reward)

    def update_policy(self):
        returns = []
        G = 0
        for r in reversed(self.rewards):
            G = r + self.gamma * G
            returns.insert(0, G)
        returns = torch.tensor(returns, dtype=torch.float32)

        states = torch.stack(self.states)
        log_probs = torch.stack(self.log_probs)
        values = self.value_net(states)

        advantages = returns - values.detach()

        # Policy loss
        policy_loss = -torch.sum(log_probs * advantages)

        # Value loss (MSE)
        value_loss = nn.functional.mse_loss(values, returns)

        self.policy_optimizer.zero_grad()
        policy_loss.backward()
        self.policy_optimizer.step()

        self.value_optimizer.zero_grad()
        value_loss.backward()
        self.value_optimizer.step()

        self.log_probs = []
        self.rewards = []
        self.states = []

        return policy_loss.item(), value_loss.item()

```

Figure 5.5: MC REINFORCE with Baseline: Algorithm Class

5.2 DUELING DQN

```
class DuelingDQN_V1_Softmax:
    def __init__(self, model, env: gym.Env, lr: float, initial_tau: float, tau_decay: float, final_tau: float, disc_factor: float = 0.99):
        self.model = model
        self.env = env
        self.lr = lr
        self.tau = initial_tau
        self.tau_decay = tau_decay
        self.final_tau = final_tau
        self.disc_factor = disc_factor
        self.training_error = []

    def get_action(self, obs: np.ndarray) -> int:
        obs = torch.FloatTensor(np.array(obs)).unsqueeze(0).to(device)
        q_values = self.model(obs).detach().cpu().numpy().flatten()
        exp_q = np.exp(q_values - np.max(q_values))
        action_probabilities = exp_q / np.sum(exp_q)
        return np.random.choice(self.env.action_space.n, p=action_probabilities)

    def decay_tau(self):
        self.tau = max(self.final_tau, self.tau * self.tau_decay)
```

Figure 5.6: Dueling DQN Base class

```
class QNetwork_model(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(QNetwork_model, self).__init__()

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.fc1 = nn.Linear(self.input_dim, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc_value = nn.Linear(64, 256)
        self.fc_advantage = nn.Linear(64, 256)
        self.value_layer = nn.Linear(256, 1)
        self.advantage_layer = nn.Linear(256, self.output_dim)
        self.relu = nn.ReLU()

    def forward(self, x):
        y = self.relu(self.fc1(x))
        y = self.relu(self.fc2(y))
        value = self.relu(self.fc_value(y))
        value = self.value_layer(value)
        advantage = self.relu(self.fc_advantage(y))
        advantage = self.advantage_layer(advantage)
        q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))

        return q_values
```

Figure 5.7: Dueling DQN Version 1 Q network class

```
class QNetwork_model_2(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(QNetwork_model_2, self).__init__()

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.fc1 = nn.Linear(self.input_dim, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc_value = nn.Linear(64, 256)
        self.fc_advantage = nn.Linear(64, 256)
        self.value_layer = nn.Linear(256, 1)
        self.advantage_layer = nn.Linear(256, self.output_dim)
        self.relu = nn.ReLU()

    def forward(self, x):
        y = self.relu(self.fc1(x))
        y = self.relu(self.fc2(y))
        value = self.relu(self.fc_value(y))
        value = self.value_layer(value)
        advantage = self.relu(self.fc_advantage(y))
        advantage = self.advantage_layer(advantage)
        q_values = value + (advantage - advantage.max(dim=1, keepdim=True)[0])

        return q_values

model = QNetwork_model_2(env.observation_space.shape[0], int(env.action_space.n))
```

Figure 5.8: Dueling DQN Version 2 Q network class

GitHub Link: https://github.com/DA6400-RL-JanMay2025/programming-assignment-02-me21b037_me21b122

Credits and Resources

- OpenAI GPT-4-turbo - For Latex boilerplate code